

Përgjigje me Shkrim - Kërkesa 10

Punuar nga: Dion Salihu

Tema: JWT Microservices Authentication System

(a) JWT vs Session

Session-based authentication ruan gjendjen e përdoruesit në server. Kur përdoruesi bën login, serveri krijon një session ID dhe zakonisht e ruan në cookie. Në çdo request tjetër, serveri duhet ta kontrollojë atë session në memorie, cache ose databazë.

Kjo qasje është e thjeshtë për aplikacione monolitike, por bëhet më komplekse në mikrosërbbime. Nëse sistemi ka disa services, atëherë të gjitha duhet të kenë qasje te i njëjti session storage ose duhet të përdoren sticky sessions. Kjo e rrit varësinë midis shërbimeve.

JWT authentication është stateless. Pas login-it, Auth Service krijon një token të nënshkruar dhe ia kthen klientit. Klienti e dërgon token-in në çdo request:

Authorization: Bearer <JWT_TOKEN>

Secili Resource Service mund ta validojë JWT vetë duke kontrolluar:

- nënshkrimin
- issuer
- audience
- skadimin e token-it

Në këtë projekt, **auth-service** lëshon JWT, ndërsa **resource-service-1** dhe **resource-service-2** e validojnë token-in me të njëjtin sekret. Kjo e bën JWT të përshtatshëm për mikrosërbbime, sepse Resource Services nuk kanë nevojë ta pyesin Auth Service për çdo request.

(b) bcrypt vs SHA-256

SHA-256 është algoritëm hash shumë i shpejtë dhe deterministik. Ai është i dobishëm për integritetin e të dhënave, për shembull për të kontrolluar nëse një file është ndryshuar. Por për password-e nuk është zgjidhje e mirë, sepse shpejtësia e tij i ndihmon sulmuesit të provojnë shumë kombinime në kohë të shkurtër.

bcrypt është krijuar posaçërisht për ruajtjen e password-eve. Ai përdor salt unik për çdo password dhe ka work factor, që e bën procesin më të ngadalshëm. Kjo e vështirëson brute-force offline.

Në këtë projekt, gjatë regjistrimit përdoret:

```
BCrypt.Net.BCrypt.HashPassword(dto.Password)
```

Gjatë login-it përdoret:

```
BCrypt.Net.BCrypt.Verify(dto.Password, user.PasswordHash)
```

Kjo do të thotë që password-i real nuk ruhet në databazë. Në databazë ruhet vetëm hash-i.

(c) Isolation Forest

Isolation Forest është algoritëm unsupervised për anomaly detection. Ai ndërton pemë rastësore dhe mat sa shpejt izolohet një pikë. Pikat normale zakonisht kërkojnë më shumë ndarje për t'u izoluar, ndërsa pikat e pazakonta izolohehen më shpejt.

Ky algoritëm është i përshtatshëm për login security sepse shpesh nuk kemi shumë shembuj realë të sulmeve të etiketuara. Në vend të kësaj, mund të trajnojmë modelin mbi sjellje normale dhe disa shembuj sintetikë anomalish.

Në këtë projekt, `ai-service/train.py` trajnon një model `IsolationForest` mbi features të login-it si:

- ora e login-it
- tentimet e dështuara
- IP e re
- success rate
- frekuenca e login-it
- ndryshim shteti
- impossible travel
- distanca në kilometra
- koha nga login-i i fundit
- ndryshim user-agent/browser

Modeli ruhet në:

```
models/isolation_forest.pkl
```

(d) Pse rate limiting nuk mjafton

Rate limiting kufizon numrin ose shpejtësinë e request-ve. Për shembull, mund të lejojë vetëm disa tentativa login-i brenda një minute. Kjo ndihmon kundër brute-force të thjeshtë.

Megjithatë, rate limiting nuk mjafton për të gjitha rastet:

- credential stuffing mund të bëhet ngadalë, pa e kaluar limitin
- login nga një lokacion i pamundur nuk zbulohet vetëm nga numri i request-ve
- një login në orë të pazakontë mund të jetë i dyshimtë edhe nëse është vetëm një request
- ndryshimi i IP-së, shtetit dhe browser-it kërkon analizë konteksti

Prandaj AI anomaly detection mund të analizojë disa features së bashku, jo vetëm numrin e request-ve. Në këtë projekt, Isolation Forest trajnohet mbi features të login-it për të kuptuar dallimin mes sjelljes normale dhe sjelljes së pazakontë.

(e) Krahësim me Auth0 / Keycloak / Cognito

Platforma	Përshkrimi	Përparësitë	Kufizimet
Auth0	SaaS identity provider	Setup i shpejtë, OAuth/OIDC, dashboard i gatshëm	Varësi nga palë e tretë, kosto
Keycloak	Open-source identity provider	Kontroll i lartë, self-hosted, enterprise features	Setup dhe mirëmbajtje më komplekse
AWS Cognito	Shërbim i AWS për auth	Integrim i mirë me AWS, menaxhim cloud	I lidhur me ekosistemin AWS
Ky projekt	Implementim edukativ custom	Tregon JWT, bcrypt, mikroshërbime dhe AI training në kod	Jo production-ready pa hardening shtesë

Ky projekt nuk synon të zëvendësojë Auth0, Keycloak ose Cognito në prodhim. Qëllimi është edukativ: të demonstrohet se si funksionon autentifikimi JWT në mikroshërbime dhe si mund të trajnohet një model AI për login anomaly detection.

Përfundim

Përgjigjet mbulojnë konceptet kryesore të projektit: JWT, sessions, bcrypt, SHA-256, Isolation Forest, rate limiting dhe krahasimin me platforma ekzistuese të autentifikimit. Këto koncepte lidhen direkt me implementimin në kod.